

Chunking Strategy Benchmark Report

By Insharah Irfan Nazir

1. Introduction.....	1
2. Implemented Chunking Strategies.....	1
Fixed-Size Chunking.....	1
Recursive Chunking.....	2
Semantic Chunking.....	2
3. Benchmark Methodology.....	3
4. Benchmark Results.....	3
4.1 Runtime.....	4
4.2 Chunk Size Distribution.....	4
4.3 Boundary Quality.....	5
4.4 Semantic Coherence.....	6
4.5 Speed vs. Quality Trade-off.....	6
5. Discussion.....	7
6. Recommendations.....	7
7. Conclusion.....	8

1. Introduction

Chunking is the process of splitting long documents into smaller, self-contained units suitable for embedding, retrieval, and downstream LLM processing. In a document ingestion pipeline, raw documents are too long to process as a single unit — most embedding models have a maximum input length (typically 512 tokens), and retrieval quality degrades when a chunk contains multiple unrelated topics. This benchmark compares three chunking strategies — fixed-size, recursive, and semantic — across a multilingual corpus to evaluate the trade-offs between processing speed, structural quality, and semantic coherence.

2. Implemented Chunking Strategies

Fixed-Size Chunking

Concept: Divide the tokenized text into windows of a fixed token count, with a configurable overlap between consecutive windows so that context is not completely lost at chunk boundaries.

Implementation: Text is tokenized using OpenAI's `cl100k_base` tokenizer (via `tiktoken`), which provides a token count consistent with GPT-3.5/4-era models. A sliding window of 512 tokens advances by 462 tokens at each step ($512 - 50$ token overlap), producing chunks of nearly uniform size. The final chunk may be shorter than 512 tokens if the document's token count does not divide evenly.

Parameters: `chunk_size=512, overlap=50`

Design rationale: Fixed-size chunking makes no assumptions about the document's content or structure. It is the simplest possible strategy and the natural baseline against which the other two are compared. Its primary advantage is speed — tokenization and slicing are computationally trivial.

Known limitations: It treats the text as a raw byte stream, cutting at wherever a token count happens to land, regardless of sentence or paragraph boundaries.

Recursive Chunking

Concept: Split the document on the largest structural separator available, then recurse into smaller separators only for pieces that are still too large to fit within the chunk size limit. This produces chunks that respect natural text boundaries wherever the content allows, falling back to harder splits only when necessary.

Implementation: A separator hierarchy is applied in order from largest to smallest: paragraph breaks (`\n\n`), line breaks (`\n`), sentence ends (`.`), word boundaries (`\s`), and finally individual characters. The text is first flattened into atomic pieces — each individually guaranteed to fit within the 512-token limit — using recursive splitting. A separate, non-recursive packing pass then greedily merges atomic pieces into final chunks up to the size limit, carrying a bounded overlap tail (50 tokens) between consecutive chunks.

Parameters: `chunk_size=512, overlap=50, separator_hierarchy ["\n\n", "\n", ".", " ", " ", ""]`

Design rationale: Recursive splitting and packing are deliberately kept as separate phases. An earlier interleaved implementation caused overlap to be silently discarded whenever an oversized piece needed further splitting, because the recursive call handling the split started with an empty buffer and lost the parent call's overlap state. Separating the two phases ensures overlap carries correctly across every chunk boundary, including those originating from splitting an oversized paragraph.

Known limitations: Chunk quality is sensitive to how consistently the source document uses paragraph/line breaks; text with little structural markup (e.g. dialogue-heavy drama) degrades toward fixed-size-like performance.

Semantic Chunking

Concept: Place chunk boundaries where the meaning of the text actually shifts — where consecutive sentences are less similar to each other — rather than at a fixed token count or structural separator. This produces chunks that are internally coherent by topic rather than by size or structure alone.

Implementation: The text is split into sentences using a regex that matches sentence-ending punctuation across multiple scripts: Latin (`., !, ?`), Urdu (`-`), Arabic (`؟`), and full-width variants (`！, 。, 。`). Each sentence is embedded individually using sentence-transformers with the `all-MiniLM-L6-v2` model. Cosine similarity is computed between each consecutive pair of sentence embeddings. A dynamic threshold, set one standard deviation below the document's mean pairwise similarity, triggers a chunk boundary when similarity drops below it.

Three guardrails prevent degenerate output:

- A **hard token ceiling** (512 tokens) force-flushes the current chunk regardless of similarity.
- A **minimum token floor** (128 tokens) prevents noisy short sentences — common in OCR output and structured documents — from fragmenting into many tiny chunks.
- **Oversized single sentences** (run-on prose or OCR-garbled text missing punctuation) are pre-split using token-window slicing before embedding, so they cannot bypass the ceiling check.

Parameters: chunk_size=512, overlap=50, std_multiplier=1.0, min_chunk_tokens=128

Design rationale: The overlap mechanism uses a direct token-level slice of the already-assembled chunk text as a seed for the next chunk. The overlap seed is post-processed to guarantee it ends on a recognized sentence-terminating character before being prepended to the next chunk, ensuring any downstream process that re-splits chunk text into sentences (including the coherence evaluation in this benchmark) cannot silently fuse the seed with the chunk's first real sentence.

3. Benchmark Methodology

Dataset

- 60 documents (20 PDF, 20 DOCX, 20 HTML)
- Languages: English, French, Urdu, Korean, Arabic
- Includes both native-text and OCR-extracted documents
- Content types: English Technical (51), Other (45), English Literary (30), Urdu (21), Arabic/Other Scripts (15), French (12), Korean (3), Exam/Structured (3)

Metrics

- Runtime per document
- Chunk size distribution (mean and standard deviation, in tokens)
- Boundary quality — fraction of chunks ending at a real sentence terminator, used as a proxy for structural coherence
- Semantic coherence — average cosine similarity between sentences within a chunk vs. across a chunk boundary, computed on the first 10 documents of the corpus

Common Parameters

- Chunk size: 512 tokens
- Overlap: 50 tokens

4. Benchmark Results

Metric	Fixed	Recursive	Semantic	Best	Notes
Avg time/document	0.047 s	0.35 s	18.2 s	Fixed	Semantic embedding dominates runtime
Total chunks	8,351	10,048	13,800	—	Semantic creates finer chunks
Avg chunk size	499.2	446.3	319.7	Fixed	Higher token-budget utilization
Std. deviation	42.9	68.7	129.9	Fixed	Semantic most variable
Boundary quality	4.6%	38.6%	92.7%	Semantic	Primary quality metric
Intra similarity	0.301	0.301	0.3022	—	Nearly identical
Cross similarity	0.302	0.300	0.1629	Semantic	Clear gap

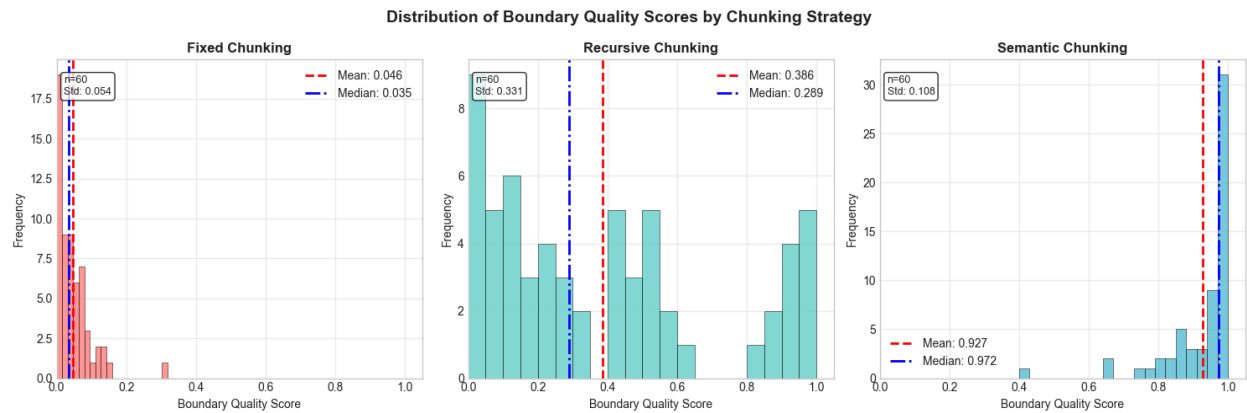
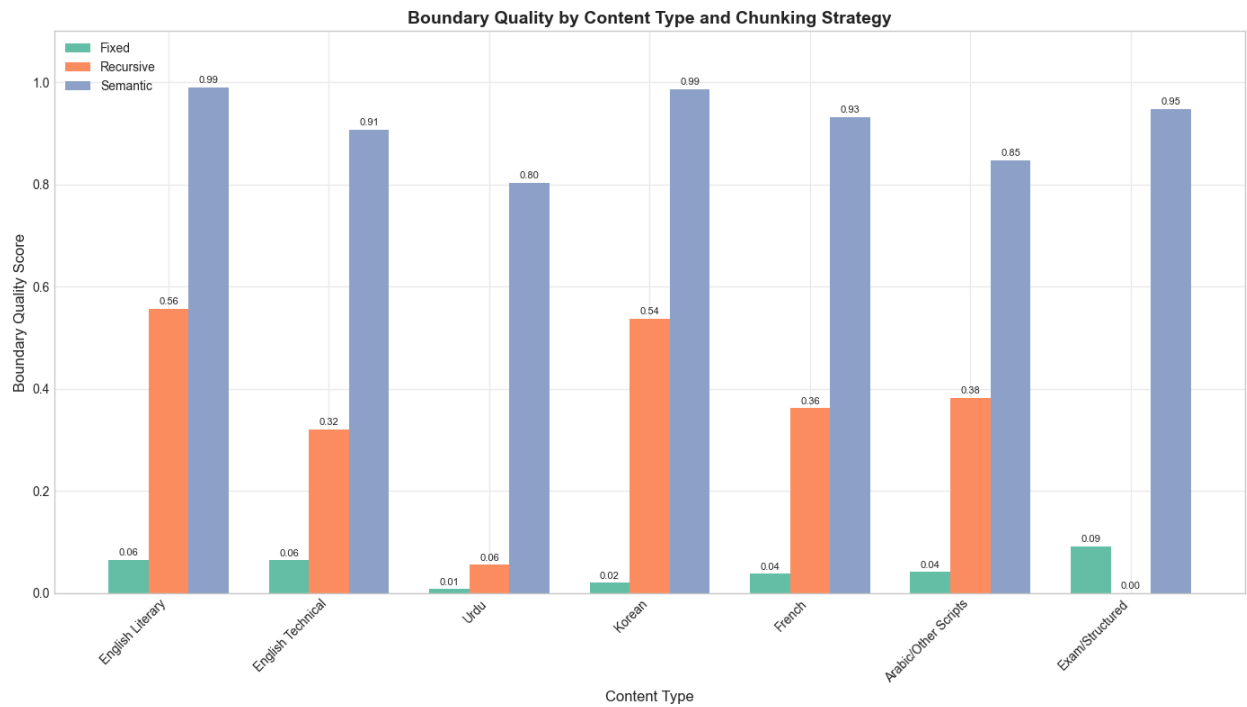
4.1 Runtime

Fixed-size chunking is the fastest by a wide margin — tokenization and slicing are computationally trivial, averaging 0.047 seconds per document. Recursive chunking is roughly 7x slower (0.35s/doc) but remains well within the range of practical batch processing. Semantic chunking is 384x slower than fixed-size (18.2s/doc) because every sentence in every document must be individually embedded by a neural encoder running on CPU. Across the full 60-document corpus, semantic chunking took roughly 18 minutes total; extrapolated to 1,000 documents, that's approximately 5 hours on CPU, versus under 6 minutes for recursive. GPU acceleration would reduce semantic's per-document cost to an estimated 1–2 seconds, making it practical at batch scale.

4.2 Chunk Size Distribution

Fixed-size chunking produces the most uniform chunks (499.2 ± 42.9 tokens) by design, with variance arising only from each document's final, partially-filled chunk. Recursive chunking averages somewhat smaller chunks (446.3 ± 68.7 tokens) because it stops at structural boundaries rather than packing to the token limit — some budget is left unused at the end of a paragraph or sentence rather than filled with content from the next section. Semantic chunking is the most variable (319.7 ± 129.9 tokens): chunks range from the 128-token floor (short, single-topic passages) up to the 512-token ceiling (long passages with no detected topic shift). This reflects semantic chunking placing more boundaries overall — topic shifts are detected within paragraphs, not just between them — producing more, smaller chunks (13,800 vs. 10,048 recursive vs. 8,351 fixed). Both recursive and semantic prioritize coherent boundaries over fully utilizing the token budget.

4.3 Boundary Quality



Boundary quality is the strongest differentiating metric in this benchmark. Fixed-size chunking lands on a real sentence boundary in only 4.6% of chunks — fewer than 1 in 20 — since it has no awareness of where sentences end. On literary prose (*Of Mice and Men*, *The Stranger*) it scored 5–6%; on drama (*Macbeth*) and exam papers it dropped to 0%.

Recursive chunking improves this to 38.6%, a 9x gain, by respecting the paragraph/sentence separator hierarchy. Results vary substantially by content type: dialogue-heavy drama and structured DOCX files scored 0–0.1, while clean prose-format DOCX files (e.g. *Albert Camus.docx*, *ANIMAL FARM.docx*) scored 0.989–1.0. This wide variance reflects the separator hierarchy's sensitivity to how consistently the source text uses paragraph breaks.

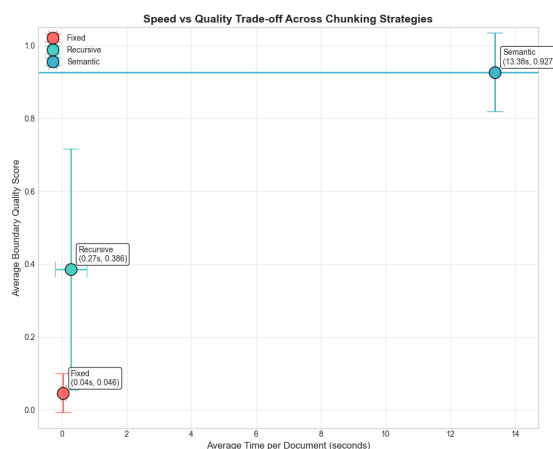
Semantic chunking reaches 92.7% — a 20x improvement over fixed-size and 2.4x over recursive — and its high performance is consistent across content types: literary prose 0.97–1.0, Korean OCR 0.986, French 0.93–1.0, Urdu native text 0.966, Arabic Wikipedia 0.65–0.97. Its only weak spot is very short documents (under 128 words), where the minimum token floor prevents fine-grained splitting. For a downstream RAG or LLM pipeline, boundary quality has a direct impact on retrieval quality: a chunk that ends mid-sentence forces the model to work from incomplete context, degrading both precision and generation coherence.

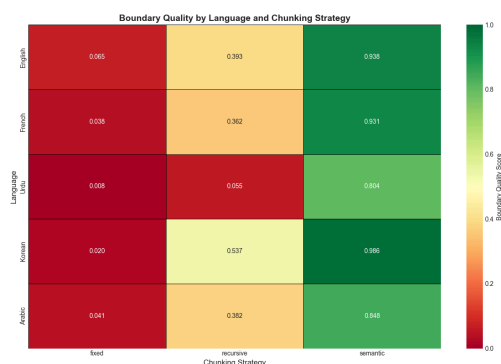
4.4 Semantic Coherence

Semantic splitting wins at semantic coherence; intra-chunk similarity (0.302) is clearly above cross-boundary similarity (0.163) for semantic chunking. This is the signature of a chunker correctly placing boundaries at genuine topic shifts: sentences within a chunk are meaningfully more related to each other than to whatever follows the cut. Fixed and recursive chunking, whose boundaries aren't semantically informed, show no such gap (0.301 vs. 0.302, and 0.301 vs. 0.300 respectively) — as expected, since neither strategy optimizes for this property.

4.5 Speed vs. Quality Trade-off

The scatter plot makes the central trade-off of this benchmark immediately visible: fixed-size is fast and low-quality, recursive is moderate on quality yet fast, and semantic is slow but high-quality.





The per-language heatmap supports the routing recommendations in Section 6 by showing that semantic chunking's advantage holds consistently across languages and scripts, not just in English.

5. Discussion

Boundary Quality

Boundary quality was the single most discriminative metric in this benchmark. The gap between strategies — 4.6% (fixed) → 38.6% (recursive) → 92.7% (semantic) — is large enough to be the primary basis for strategy selection, since it maps directly onto how usable a retrieved chunk is for a downstream model.

Runtime vs. Quality Trade-off

No strategy wins on both axes. Fixed-size chunking is essentially free computationally but produces the lowest-quality chunks. Semantic chunking produces the best boundaries but is 384x slower than fixed-size on CPU, which limits it to interactive, single-document use cases in the current deployment. Recursive chunking sits in between on both dimensions and is the practical default when neither extreme quality nor extreme throughput is required.

Effect of OCR Quality

Urdu OCR documents produced consistently lower boundary quality scores than native-text documents of comparable length and language, across all three strategies. Tesseract's Nastaliq OCR accuracy is documented at 35–55% confidence for Urdu, meaning sentence-ending characters are frequently misread or dropped. With no clean sentence terminators to detect, even semantic chunking is working from degraded signal. This reinforces the pipeline's OCR investigation findings: improving Urdu OCR accuracy is the highest-leverage improvement available for the Urdu portion of the pipeline — more so than any change to chunking strategy.

6. Recommendations

Document Type	Recommended Strategy	Reason

English/French literary	Semantic	Highest boundary quality (0.97–1.0); recursive fails on dialogue-heavy text (e.g. <i>Macbeth</i> : 0.042 recursive vs. 0.991 semantic)
Technical reports	Recursive	Good quality-speed balance; gap to semantic is smaller than on literary text (0.08–0.56 vs. 0.83–0.98), and recursive is 7x faster
Urdu native text	Semantic	0.966 boundary quality after extending the sentence splitter for -; recursive manages only 0.055
Urdu OCR	Semantic	Best available result (0.905), but OCR quality is the real bottleneck
Korean/Arabic	Semantic	Outperforms recursive decisively (0.986 vs. 0.537 Korean; 0.65–0.97 vs. 0.12–0.53 Arabic)
Large batch ingestion	Recursive	High throughput (~6 min for 1,000 docs vs. ~5 hrs for semantic on CPU)

7. Conclusion

Fixed-size chunking provides a fast baseline but poor boundary quality. Recursive chunking substantially improves structural coherence while remaining computationally efficient, making it the practical default for high-throughput batch processing. Semantic chunking consistently produces the highest-quality boundaries — over 92% sentence-aligned across a multilingual corpus — at the expense of significantly higher runtime on CPU. The benchmark demonstrates that no single strategy is optimal for every workload: the appropriate choice depends on document type, language, and throughput requirements, with semantic chunking recommended for quality-sensitive interactive use and recursive chunking recommended for high-volume batch ingestion.